Object Oriented Programming with Java (BCS-403)

Unit 3: Java New Features

Complete Notes

Topics Covered

Functional Interfaces, Lambda Expressions, Method References, Stream API, Default Methods, Static Methods, Base64 Encode and Decode, ForEach Method, Try-with-resources, Type Annotations, Repeating Annotations, Java Module System, Diamond Syntax with Inner Anonymous Class, Local Variable Type Inference, Switch Expressions, Yield Keyword, Text Blocks, Records, and Sealed Classes.

Prepared from Unit III source material

Contents

1. Functional Interfaces

Theory: A functional interface is an interface that contains exactly one abstract method. It is often referred to as **SAM** (Single Abstract Method). Introduced in Java 8, functional interfaces form the foundation for bringing functional programming concepts into Java. They provide the target type for lambda expressions.

Key Rules and Characteristics

- It can have only one unimplemented abstract method. This single method represents the single functionality the interface exhibits.
- It can contain any number of `default` or `static` methods without losing its functional status, because those methods already have bodies.
- The `@FunctionalInterface` annotation is optional but highly recommended. It forces the compiler to strictly check the interface. If a programmer accidentally adds a second abstract method, the compiler throws an error.

Standard Examples: `Runnable` (contains only `run()`) and `Comparator` (contains only `compare()`).

Example:

```
@FunctionalInterface
interface Calculator {
    int sum(int a, int b);   // The single abstract method

    // Adding default/static methods does not violate the rule
    default void printInfo() {
        System.out.println("This is a functional interface.");
    }
}
```

2. Lambda Expressions

Theory: A lambda expression is a concise way to represent an anonymous function, that is, a method that does not have a name, a return type declaration, or an access modifier such as `public` or `private`.

Key Rules and Characteristics

- **Reduces Boilerplate:** It eliminates the need to write bulky anonymous inner classes.
- **Functional Style:** It allows functions to be passed as parameters to other methods or stored in variables.
- **Type Inference:** The Java compiler can infer parameter types from the target functional interface.

General Syntax:

```
(parameters) -> { body / expression }
```

The arrow `->` separates parameters from implementation.

Example:

```java
public class LambdaExample {
    public static void main(String[] args) {
        // Implementing the Calculator interface using a lambda expression
        // Notice we did not specify "int a, int b" - the compiler infers it
        Calculator c = (a, b) -> a + b;

        System.out.println("Addition Result: " + c.sum(10, 20)); // Output: 30
    }
}
```

3. Method References

Theory: A method reference is a shorter and more compact shorthand syntax that allows you to refer to an existing method by name instead of invoking it through a lambda expression. If the lambda body only calls an existing method, the lambda can often be replaced with a method reference.

Key Rules and Characteristics

- It uses the double-colon operator `::`.
- It improves readability, promotes code reuse, and maintains type safety.

3.1 Three Types of Method References

- **Reference to a static method:** `ClassName::staticMethodName`
- **Reference to an instance method of an object:** `instanceReference::methodName`
- **Reference to a constructor:** `ClassName::new`

Example 1: Static Method Reference

```
import java.util.function.Function;

public class MethodRefDemo {
    public static void main(String[] args) {
        // 1. Traditional lambda expression approach
        Function<String, Integer> lambda = s -> Integer.parseInt(s);

        // 2. Static method reference approach (shorter and cleaner)
        Function<String, Integer> methodRef = Integer::parseInt;

        System.out.println(methodRef.apply("100")); // Converts String to int
    }
}
```

Example 2: Constructor Reference

```
import java.util.function.Supplier;
import java.util.List;
import java.util.ArrayList;

public class ConstructorRef {
    public static void main(String[] args) {
        // Using lambda expression
        Supplier<List<String>> listSupplierLambda = () -> new ArrayList<>();

        // Using constructor method reference (cleaner)
        Supplier<List<String>> listSupplierMethodRef = ArrayList::new;

        List<String> myList = listSupplierMethodRef.get(); // Creates the object
    }
```

```
}
```

4. Default and Static Methods in Interfaces

Before Java 8, interfaces could only contain abstract methods. That created a major limitation: adding a new method to an old interface would break all existing implementing classes.

4.1 Default Methods

Theory: Default methods are methods defined inside an interface with a full body and marked with the `default` keyword.

Purpose (Backward Compatibility): They allow developers to add new functionality to existing interfaces without breaking old implementations. Implementing classes inherit the default method automatically and may override it if custom behavior is needed.

Multiple Inheritance Conflict: If a class implements two interfaces that both define a default method with the same name and signature, the class must override that method to resolve the ambiguity.

4.2 Static Methods

Theory: Static methods are methods defined using the `static` keyword inside an interface.

Rule: Because they are static, they belong to the interface itself. They cannot be overridden by implementing classes, and they must be called using the interface name.

Example:

```
interface Vehicle {
    void start(); // Abstract method

    default void honk() { // Default method (can be overridden)
        System.out.println("Honking the horn...");
    }
    static void service() { // Static method (cannot be overridden)
        System.out.println("Servicing the vehicle...");
    }
}
```

4.3 Difference Between Default and Static Methods

Feature	Default Method	Static Method
Definition	A method in an interface that has a body and uses the `default` keyword	A method in an interface declared using the `static` keyword
Introduced	Java 8	Java 8
Purpose	To provide a default implementation to implementing classes	To provide utility or helper methods related to the interface
Keyword used	`default`	`static`

Implementation	Written inside the interface with method body	Written inside the interface with method body
Access	Called using the object of the implementing class	Called using the interface name
Overriding	Can be overridden by implementing classes	Cannot be overridden by implementing classes
Inheritance	Inherited by implementing classes	Not inherited by implementing classes
Example call	`obj.methodName()`	`InterfaceName.methodName()`

5. Stream API

Theory: Found in the `java.util.stream` package, a stream is a sequence of elements from a collection such as a `List` or array that supports functional-style, sequential, and parallel aggregate operations.

Core Concepts

- Streams do not store data; they compute data on demand.
- They heavily use lambda expressions to process datasets efficiently without traditional explicit loops.

5.1 Types of Operations

Intermediate Operations: These transform, filter, or manipulate a stream and return a new stream. They are lazy and execute only when a terminal operation is called.

Examples: `filter()`, `map()`, `sorted()`, `distinct()`.

Terminal Operations: These produce a final result or side effect and close the stream.

Examples: `collect()`, `reduce()`, `forEach()`, `count()`.

Example 1: Filter, Map, and Collect

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

        // Task: filter even numbers, double them, and collect into a new list
        List<Integer> result = numbers.stream()
            .filter(n -> n % 2 == 0)  // Intermediate operation (filters 2, 4, 6)
            .map(n -> n * 2)        // Intermediate operation (doubles to 4, 8, 12)
            .collect(Collectors.toList()); // Terminal operation (gathers result)
        System.out.println(result); // Output: [4, 8, 12]
    } }
```

Example 2: Stream Creation

```
import java.util.stream.Stream;
import java.util.Arrays;
import java.util.List;

public class StreamCreation {
    public static void main(String[] args) {
        // 1. Empty stream
        Stream<String> streamEmpty = Stream.empty();

        // 2. Stream from collection
        List<String> collection = Arrays.asList("a", "b", "c");
        Stream<String> streamCollection = collection.stream();

        // 3. Stream from array
        String[] arr = {"a", "b", "c"};
        Stream<String> streamOfArrayFull = Arrays.stream(arr);
        Stream<String> streamOfArrayPart = Arrays.stream(arr, 1, 3); // Part of
    array

        // 4. Stream using builder
        Stream<String> streamBuilder = Stream.<String>builder().add("a").add("b").
    build();

    // 5. Stream using generate and iterate (infinite streams limited to a size)
        Stream<String> streamGenerated = Stream.generate(() -> "element").limit(10)
    ;
        Stream<Integer> streamIterated = Stream.iterate(40, n -> n + 2).limit(20);
    } }
```

Example 3: Calculating Average using `mapToInt`

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

double average = numbers.stream()
    .mapToInt(Integer::intValue) // Converts Integer wrapper to primitive int
    .average()                   // Calculates average
    .orElse(0);                  // If stream is empty, return 0

System.out.println("Average: " + average);
```

Example 4: Finding the Longest String using `max`

```
List<String> strings = Arrays.asList("apple", "banana", "strawberry", "kiwi");
Optional<String> longestString = strings.stream()
    .max((s1, s2) -> s1.length() - s2.length()); // Custom lambda comparator
longestString.ifPresent(str -> System.out.println("Longest string: " + str));
```

6. Base64 Encode and Decode

Theory (The Problem): Computers process binary data. When raw binary data such as images or files is sent over text-only protocols like email or HTTP, direct transmission can create problems because special bytes may be interpreted incorrectly.

The Solution: Base64 encoding converts raw binary data into a safe textual format. It maps 3 bytes of binary data (24 bits) into 4 readable ASCII characters (6 bits each, giving $2^6 = 64$ characters). The size becomes about 1.33 times the original.

Padding (=): If the data length is not a multiple of 3 bytes, the encoder adds one or two padding characters = at the end. This can be skipped using `withoutPadding()`.

Example 1: Encode and Decode

```
import java.util.Base64;

public class Base64Demo {
    public static void main(String[] args) {
        String data = "AKTU Exam Data";
        byte[] bytes = data.getBytes();

        // 1. Encoding (binary to text)
        String encoded = Base64.getEncoder().encodeToString(bytes);
        System.out.println("Encoded Text: " + encoded);

        // 2. Decoding (text back to binary)
        byte[] decodedBytes = Base64.getDecoder().decode(encoded);
        System.out.println("Decoded String: " + new String(decodedBytes));
    }
}
```

Example 2: Without Padding

```
import java.util.Base64;

public class Base64PaddingDemo {
    public static void main(String[] args) {
        byte[] b = "Example of Base 64 Conversion".getBytes();

        // Skipping the extra padding '=' characters at the end
        String base64 = Base64.getEncoder().withoutPadding().encodeToString(b);
        System.out.println(base64);
    }
}
```

7. ForEach Method

Theory: Introduced in Java 8, `forEach` provides a concise way to iterate over elements of a collection such as a `List`, `Set`, or `Map`.

Mechanism: It is an internal iterator defined inside the `Iterable` interface. It removes the need for traditional `for` or `while` loops. It accepts a single `Consumer` functional interface, usually supplied through a lambda expression, and performs the specified action on each element sequentially.

Example:

```
import java.util.Arrays;
import java.util.List;

public class ForEachDemo {
    public static void main(String[] args) {
        List<String> items = Arrays.asList("Apple", "Banana", "Cherry");

        // Using forEach with a lambda expression to print elements
        items.forEach(item -> System.out.println("Fruit: " + item));
    }
}
```

8. Try-with-resources

Theory: Introduced in Java 7, try-with-resources is an automatic resource management feature. Previously, programmers had to write a `finally` block and manually call `close()` on resources such as database connections, file readers, or sockets. Forgetting that caused resource leaks.

Mechanism: The resource is declared inside the parentheses immediately following the `try` keyword. The JVM guarantees that every declared resource is automatically closed at the end of the block, whether an exception occurs or not.

Rule: The object must implement `java.lang.AutoCloseable` or `java.io.Closeable`.

Example:

```
import java.io.BufferedReader;
import java.io.FileReader;

public class TryResourceDemo {
    public static void main(String[] args) {
        // BufferedReader is automatically closed after the block executes
        try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
            System.out.println(br.readLine());
        } catch (Exception e) {
            System.out.println("Exception Occurred: " + e.getMessage());
        }  // No finally block required
    } }
```

9. Type Annotations

Theory: Annotations (`@`) are tags that provide metadata to the compiler, JVM, or development tools. Before Java 8, annotations could be applied only to declarations such as classes or methods.

Java 8 Upgrade: Java 8 expanded this so annotations can now be applied anywhere a type is used. This includes object creation, type casting, generic parameters, and method parameters.

Purpose: It allows stricter compile-time checking and greater precision, for example in catching null-related issues earlier.

Example:

```
// Examples of type annotations ensuring strict compiler checks
List<@NonNull String> names;     // Ensures the list never accepts a null string
Arrays<@NonNegative Integer> data; // Ensures data is always positive
```

10. Repeating Annotations

Theory: Before Java 8, the same annotation could not be applied to a single declaration more than once. If a method needed two schedules, writing `@Schedule` twice was not allowed.

Java 8 Upgrade: Repeating annotations make it possible to apply the same metadata multiple times with different values.

Implementation Rule:

1. Create the base annotation and mark it with `@Repeatable`.
2. Create a container annotation that holds an array of repeated values.

Example:

```
import java.lang.annotation.Repeatable;

// 1. The container annotation
@interface Schedules { Schedule[] value(); }

// 2. The base annotation marked as repeatable
@Repeatable(Schedules.class)
@interface Schedule {
    String day();
    String time();
}

class Meeting {
    // 3. Applying the repeating annotation multiple times
    @Schedule(day = "Monday", time = "09:00 AM")
    @Schedule(day = "Friday", time = "04:00 PM")
    public void startMeeting() {}
}
```

11. Java Module System

Theory: Introduced in Java 9 as Project Jigsaw, the Java Module System was created to address the problem of large, monolithic, and heavy JAR-based applications.

What is a Module? A module is a level of aggregation higher than a package. It is a strict collection of related packages and resources.

Module Descriptor (`module-info.java`): Every module must have this file at its root.

Important Directives

- `requires`: states which other modules this module depends on.
- `exports`: makes specific internal packages accessible to the outside world.

Goals

- Reliable configuration
- Improved performance
- Scalable platform
- Prevention of accidental internal API usage through strong encapsulation

12. Diamond Syntax with Inner Anonymous Class

Theory: The diamond operator `<>` was introduced in Java 7 to simplify generic class instantiation. It allows the compiler to infer the type argument from the left-hand side of the assignment.

Example:

```
List<String> list = new ArrayList<>();
```

Java 9 Enhancement: Before Java 9, using the diamond operator while creating an anonymous inner class produced a compiler error. Java 9 removed that limitation, allowing type inference for anonymous classes as well.

13. Local Variable Type Inference (`var`)

Theory: Introduced in Java 10, the `var` keyword shifts responsibility for determining a local variable's type from the developer to the compiler. The compiler infers the type from the initializer.

Strict Limitations

- It can only be used for local variables inside methods, loops, or conditional blocks.
- It cannot be used for class-level instance variables or method parameters.
- The variable must be initialized at the moment of declaration.

Example:

```
public class VarDemo {
    public static void main(String[] args) {
        var name = "AKTU";                    // Compiler infers String
        var age = 21;                         // Compiler infers int
        var list = new ArrayList<String>();   // Compiler infers ArrayList<String>
    } }
```

14. Switch Expressions and Yield Keyword

Theory (Switch Expressions): Finalized in Java 14, switch expressions modernize the traditional switch statement. They use arrow syntax `->`, evaluate to a single value, and eliminate the need for `break` because they do not fall through automatically.

Theory (Yield Keyword): `yield` is a context-specific keyword used inside a switch expression block. If a case needs multiple statements before returning a value, `yield` is used to return that value from the block.

Example:

```
public class SwitchYieldDemo {
    public static void main(String[] args) {
        int day = 3;

        // Switch acting as an expression that returns an integer
        int result = switch (day) {
            case 1, 2 -> 100; // Single line, arrow syntax (no break needed)
            case 3 -> {
                System.out.println("Processing complex multi-line logic...");
                yield 200; // 'yield' returns the value out of the curly braces
            }
            default -> throw new IllegalStateException("Invalid day");
        };

        System.out.println("Result: " + result);
    }
}
```

15. Text Blocks

Theory: Finalized in Java 15, a text block is a multi-line string literal.

Syntax: It is enclosed using triple double-quotes """.

Advantage: Writing HTML, SQL, or JSON in earlier Java versions required heavy string concatenation and explicit newline escapes such as \n. Text blocks preserve formatting, indentation, whitespace, and line breaks exactly as typed, which improves readability.

Example:

```
public class TextBlockDemo {
    public static void main(String[] args) {
        // Multi-line string preserving all indents and format
        String htmlData = """
            <html>
                <body>
                    <h1>Hello AKTU Students!</h1>
                </body>
            </html>
            """;

        System.out.println(htmlData);
    }
}
```

16. Records

Theory: Finalized in Java 16, `record` is a special kind of class designed to act as an immutable data carrier.

The Problem it Solves: Traditional POJO classes require large amounts of boilerplate code such as fields, constructors, getters, setters, `equals()`, `hashCode()`, and `toString()`.

The Record Solution: Declaring a record allows the compiler to automatically generate these methods. Record components are automatically final, which makes the data immutable.

Example 1: Basic Record

```
// Just 1 line replaces a massive 40-line POJO class!
public record Student(String name, int rollNo) { }

public class RecordDemo {
    public static void main(String[] args) {
        Student s = new Student("Raman", 403);

        System.out.println(s.name());     // Auto-generated getter
        System.out.println(s.toString()); // Auto-generated formatted string
    }
}
```

Example 2: Record with Static Member and Custom Method

```
public record Employee(String name, int salary) {
    // Adding a static member to a record
    public static String company = "Gateway";

    // You can also add custom methods if needed
    public void printCompany() {
        System.out.println("Works at: " + company);
    }
}
```

17. Sealed Classes

Theory: Finalized in Java 17, a sealed class provides strict control over inheritance hierarchies. It allows a developer to explicitly restrict which classes or interfaces are permitted to extend or implement it.

Keywords: `sealed` and `permits`.

Constraint Rules: Any subclass permitted to extend a sealed class must explicitly declare one of the following modifiers:

- **final:** The inheritance chain stops here.
- **sealed:** The subclass continues the seal and must explicitly permit its own subclasses.
- **non-sealed:** The subclass breaks the seal and can then be extended by any class.

Example:

```
// The class Shape strictly allows only Circle and Rectangle to inherit it
public sealed class Shape permits Circle, Rectangle { }

// Rule 1: Circle stops inheritance completely
final class Circle extends Shape { }

// Rule 3: Rectangle breaks the seal, anyone can inherit Rectangle now
non-sealed class Rectangle extends Shape { }

// class Triangle extends Shape { } // COMPILER ERROR: Not permitted by Shape!
```

18. Quick Recap

This unit covers the modern features that significantly changed Java after Java 8. The most important ideas are:

- Functional programming support through functional interfaces, lambda expressions, method references, `forEach`, and streams.
- Interface evolution using default and static methods.
- Safer and cleaner coding through try-with-resources, type annotations, and repeating annotations.
- Language evolution through modules, diamond operator enhancement, `var`, switch expressions, `yield`, text blocks, records, and sealed classes.